

rjs_lite 项目进展汇报

面向 AI Agent 的轻量级 JavaScript 执行 Runtime

Rust 原生实现 · 零外部依赖 · 开源项目

2026 年 6 月 30 日

团队成员与分工

何禹颀

队长 / 核心架构师

- 项目整体架构设计与技术选型
- 词法分析器 (lexer) 与语法分析器 (parser) 实现
- 解释器 (interpreter) 核心引擎开发
- 运行时值模型与 AST 数据结构设计
- 原型链、闭包、this 绑定等核心语义
- Agent 接口与结构化结果输出
- 代码审查与质量把控

李春雨

队员 / 标准库开发

- Object 标准库 (keys, values, create 等)
- Array 原型方法 (map, filter, sort 等 20+)
- String 原型方法 (split, replace 等 18+)
- 错误类型系统 (Error, TypeError 等)
- 单元测试编写与回归验证
- 数组空洞语义修复与边界情况处理

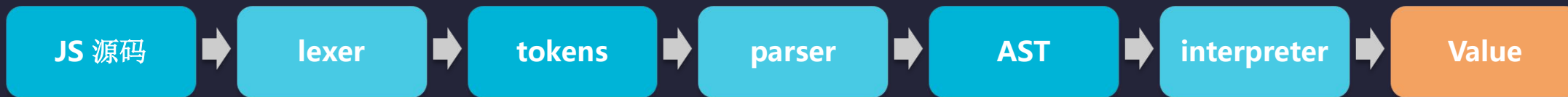
真笑君

队员 / 功能扩展 & 工程化

- REPL 交互式解释器开发与完善
- CLI 命令行入口与参数解析
- 模板字面量、箭头函数等高级语法
- Math 对象与全局函数实现
- test262 测试框架脚手架
- 项目文档编写与过程性文档
- 进展汇报材料与演示制作

项目架构与技术亮点

核心执行过程



零依赖实现

完全基于 Rust 标准库，不依赖任何第三方 crate，包体极小

非套壳引擎

自研 lexer/parser/AST/解释器，非 QuickJS/V8/Node.js 包装

Agent 优先设计

--agent-eval 模式输出结构化 JSON，含稳定 error_kind 分类

资源安全限制

源码 64KB / 步数 100K / 深度 16 层 / 输出 256 行

原型链 + 闭包

Rc<RefCell<Object>> 实现 JS 引用语义，HashMap 捕获闭包环境

丰富标准库

80+ 内置函数：
Object/Array/String/Math/Error/Function

开发时间线与任务推进

第一阶段

6/5 – 6/9

基础框架搭建

- ✓ Rust 项目脚手架初始化
- ✓ 词法分析器 (lexer) 完整实现
- ✓ 递归下降语法分析器 (parser)
- ✓ AST 全套数据结构定义
- ✓ 基础表达式与运算符优先级
- ✓ test262 测试框架集成

第二阶段

6/10 – 6/13

核心引擎开发

- ✓ 解释器核心语义实现
- ✓ 原型链与闭包环境捕获
- ✓ this 绑定与 new 构造调用
- ✓ 函数声明/表达式/调用
- ✓ 控制流 (if/while/for/switch)
- ✓ 多变量声明与成员访问

第三阶段

6/15 – 6/17

Agent 接口 & REPL

- ✓ REPL 交互式解释器 + 多行输入
- ✓ --agent-eval 结构化输出模式
- ✓ 执行步数/调用深度/输出限制
- ✓ AgentErrorKind 稳定错误分类
- ✓ break/continue 支持
- ✓ 未初始化 let 声明

第四阶段

6/23 – 6/30

标准库扩展

- ✓ Object 全套 (keys/values/create 等)
- ✓ Array 原型方法 (20+ 方法)
- ✓ String 原型方法 (18+ 方法)
- ✓ Math 对象 (全部常量与方法)
- ✓ Function.prototype (call/apply/bind)
- ✓ JSON.stringify / Error 系列

第五阶段

6/30

高级语法补充

- ✓ 模板字面量 (backtick strings)
- ✓ 箭头函数 (=>)
- ✓ 位运算符 (& | ^ ~ << >> >>>)
- ✓ for...in 循环
- ✓ delete / void / in 运算符
- ✓ 全局函数 (parseInt/parseFloat/eval)

当前成果总览

87+

Git 提交

10

核心模块

3200+

代码行数

80+

内置函数

60+

单元测试

0

外部依赖

执行引擎

- ✓ 完整 JS 执行流水线: lexer → parser → AST → interpreter
- ✓ 支持变量声明、函数、闭包、原型链、this 绑定
- ✓ 控制流全覆盖: if/while/for/switch/break/continue/try-catch

Agent 模式

- ✓ --agent-eval 结构化 JSON 输出
- ✓ 6 种稳定 error_kind 分类
- ✓ 4 维资源限制: 步数/深度/输出/源码大小

标准库

- ✓ Object 8 方法 + Array 27 方法
- ✓ String 18 方法 + Math 全部常量方法
- ✓ Error 5 类型 + Function 3 方法 + JSON

工程化

- ✓ 交互式 REPL + CLI 执行 + 文件执行
- ✓ 60+ 单元测试覆盖核心功能
- ✓ 中英文技术文档 (README + 语言子集 + Agent 契约)

JavaScript 语法支持详情

已支持

- ◉ **字面量** 数字、字符串、布尔值、null、undefined、模板字面量
- ◉ **变量声明** let、var、const, 含未初始化 let
- ◉ **运算符** 算术、比较、相等、逻辑、一元、typeof、instanceof、三元、位运算
- ◉ **赋值** 简单赋值、复合赋值 (+=, -=等)、自增自减 (++/--)
- ◉ **控制流** if/else、while、for、for...in、switch、break、continue
- ◉ **函数** 函数声明、函数表达式、箭头函数 (=>)、闭包、this、new、return
- ◉ **异常** throw、try、catch、finally
- ◉ **对象/数组** 字面量、成员访问、下标访问、delete、void、in
- ◉ **REPL** 交互式多行输入、.vars/.reset 命令

暂不支持 (后续规划)

- ◉ **ES6+ 语法**
class、module、async/await、generator、Promise
- ◉ **高级类型**
Symbol、BigInt、RegExp、Proxy、Reflect
- ◉ **完整语义**
完整的隐式类型转换、严格模式全行为
- ◉ **浏览器/Node**
DOM、File API、Network API、Process API、npm
- ◉ **性能优化**
JIT 编译、字节码 VM、高级优化流水线

Agent 工具模式详解

设计理念：为 **AI Agent** 提供本地、轻量、可嵌入的 **JavaScript** 执行工具

CLI 调用契约

```
cargo run -- --agent-eval "let x = 1 + 2; print(x); x;"
```

→ 输出单行 JSON:

```
{"ok":true,"request_id":"local","value":"3",  
  "value_type":"number","output":["3"],  
  "output_truncated":false,"error_kind":null,"error":null}
```

JSON 结果字段说明

`ok : bool` — 执行是否成功

`request_id : string` — 执行上下文 ID

`value : string|null` — 最终返回值字符串

`value_type : string` — number/string/boolean/object/undefined/function

`output_truncated : bool` — 输出是否被截断

`output : string[]` — `print()` 捕获的输出行

`error_kind : string|null` — lex/parse/runtime/source_limit/step_limit/call_depth_limit

`error : string|null` — 失败时的诊断信息

默认资源限制

- 源码大小上限：64 KiB
- 输出行数上限：256 行
- 执行步数上限：100,000
- 调用深度上限：16 层

稳定错误分类 (AgentErrorKind)

■ **lex** 词法分析错误

■ **parse** 语法分析错误

■ **runtime** 运行时错误

■ **source_limit** 源码大小超限

■ **step_limit** 执行步数超限

■ **call_depth_limit** 调用深度超限

后续规划与展望

功能完整度

- ▶ 加固对象/数组/原型/异常边界语义
- ▶ 补齐常用标准库 (RegExp, Promise 等)
- ▶ 建立 test262 子集工作流, 记录通过率
- ▶ 优先覆盖 AI Agent 高频 JS 子集
- ▶ 扩展 HostFunction 注册机制
- ▶ 完善严格模式行为

性能优化

- ▶ 引入字节码编译与 VM 执行层
- ▶ 增加微基准测试 (算术/循环/函数调用)
- ▶ 参考 JetStream 标准设计性能测试
- ▶ 增加 Agent workload 冷启动/重复脚本测试
- ▶ 优化启动时间与内存占用
- ▶ 解释器热路径性能分析

创新方向

- ▶ 保持原生 Rust 实现, 避免套壳
- ▶ 针对短生命周期脚本执行优化
- ▶ Agent 可挂载可控宿主能力
- ▶ 探索安全沙箱与资源隔离
- ▶ AgentRuntime 可定制化配置
- ▶ 支持更多宿主 API 扩展点